



VOYA COLLABORATIVE TRAVEL PLANNER

PROJECT DELIVERABLE #2

PREPARED FOR

University of Texas at Dallas
Fall 2025

CS/SE/CE 3354.008 – Software Engineering

Instructor: Dr. Barbara Maweu

PREPARED BY

Team 8 - Sanyukta Bhadlikar, Justin Castro, Oluwawemimo Jayeoba,
Albert Jimenez, Amrr Okasheh, Anish Patel, Suhani Rana

Submission Date: November 23, 2025

TABLE OF CONTENTS

1. Project Title

1.1 Overview and Purpose

2. Team & Task Delegation

2.1 Team Members and Deliverable #2 Task Distribution

2.2 Contribution Summary

3. Recap of Deliverable #1

3.1 Deliverable #1 Team and Task Delegation

3.2 Initial Assumptions for Deliverable #1

3.2.1 Assumptions made regarding the requirements

3.2.2 Assumptions made regarding the sequence diagrams

3.2.3 Assumptions made regarding the US government compliance laws for the app to operate successfully

3.3 Software Process Model (Incremental)

3.4 Feedback Received

3.4.1 Proposal Feedback

3.4.2 Deliverable #1 Feedback

3.5 Functional Requirements

3.5.1 User Authentication

3.5.2 Trip Management (Admin Only)

3.5.3 User Trip Access

3.5.4 Location Tracking

3.5.5 Trip Budgeting

3.5.6 Polling

3.5.7 Notifications

3.6 Non-functional Requirements

3.6.1 Usability

3.6.2 Performance

3.6.3 Space

3.6.4 Dependability

3.6.5 Security

3.6.6 Environmental

3.6.7 Operational

3.6.8 Developmental

3.6.9 Regulatory

3.6.10 Ethical

3.6.11 Accounting

3.6.12 Safety & Security

3.7 Use Case Diagram Overview

3.8 Sequence Diagram Overview

3.8.1 Sequence Diagram – User Registration and Login

3.8.2 Sequence Diagram – Create Trip

3.8.3 Sequence Diagram – Invite Members

3.8.4 Sequence Diagram – Add and Vote on Activities

3.8.5 Sequence Diagram – Manage Budget

3.8.6 Sequence Diagram – View Itinerary and Map

3.8.7 Sequence Diagram – Notifications and Reminders

3.9 Class Diagram Overview

3.10 Architectural Design Recap

4. Project Scheduling, Cost, Effort, and Pricing Estimation

4.1 Project Scheduling

4.1.1 Estimated Start and End Dates

4.1.2 Working Hours, Weekends, and Schedule Assumptions

4.1.3 Project Timeline (Gantt Chart)

4.2 Effort Estimation

4.2.1 Chosen Estimation Technique (Application Composition)

4.2.2 Rationale and Calculation Steps

4.3 Cost and Pricing Estimation

4.3.1 Hardware Cost Estimates

4.3.2 Software Cost Estimates

4.3.3 Personnel and Training Cost Estimates

4.3.4 Total Cost and Pricing Strategy

4.4 Staffing Plan and Project Duration

4.4.1 Staffing Plan

4.4.2 Development Duration

5. Test Plan for the Software

- 5.1 Overview of Test Strategy
- 5.2 Unit Selected for Testing (Description and Rationale)
- 5.3 Test Case Design and Expected Outcomes
- 5.4 Unit Implementation Code (Sample Method / Class)
- 5.5 Automated Unit Testing (JUnit)
- 5.6 Test Results and Observations

6. Comparison with Similar Systems

- 6.1 Overview of Existing Travel Planning Platforms
- 6.2 Comparative Feature Table (Voya vs. Competitors)
- 6.3 Strengths, Weaknesses, and Differentiators of Voya

7. Conclusion and Evaluation

- 7.1 Summary of Progress Since Deliverable #1
- 7.2 Deviations from Original Plan and Reasons
- 7.3 Lessons Learned and Challenges Faced
- 7.4 Future Work and Recommendations

8. References

9. Appendices

- 9.1 Presentation Slides Link
- 9.2 GitHub Repository Link

1. VOYA COLLABORATIVE TRAVEL PLANNER

1.1 Overview and Purpose

Voya is a collaborative travel planning app that centralizes trip organization for groups. It combines itinerary scheduling, budgeting, activity coordination, and real-time communication into one platform, removing the need for separate tools like spreadsheets and messaging apps.

The app enhances group coordination by allowing users to create trips, invite participants, and vote on destinations or activities. With budgeting tools, live maps, and push notifications, Voya promotes transparency, user engagement, and efficiency.

From a technical perspective, it applies key software engineering principles such as modular design, incremental development, and user-centered design to create a scalable, secure, and maintainable system.

Goals:

- Centralize itinerary, budget, and communication management.
- Enable collaborative decision-making through shared tools and polls
- Ensure reliability and security through authentication, encryption, and data backup
- Demonstrate professional software engineering practices from design to testing

2. TEAM AND TASK DELEGATION

2.1 Team Members and Deliverable #2 Task Distribution

Team Members	Delegated Task
Suhani Rana	Conclusion and Evaluation, Appendices, Slides + Presentation Recording and Compilation
Sanyukta Bhadlikar	Use case diagram, Objectives, Slides and Presentation Recording, Conclusion and Evaluation

Justin Castro	Slides & Presentation Recording, Conclusion and Evaluation
Oluwawemimo Jayeoba	Slides & Presentation Recording, Report + Slides Setup & Formatting, Comparison Report
Albert Jimenez	Project Scheduling, Effort Estimation, Slides & Presentation Recording
Amrr Okasheh	Test Plan for the Software, JUnit tests, Slides + Recording
Anish Patel	Project Cost and Pricing Estimation, Staffing Plan and Project Duration, and Slides + Recording

2.2 Contribution Summary ✓

Suhani Rana - I completed the Conclusion and Evaluation, Appendices, Slides + Presentation Recording and Compilation.

Sanyukta Bhadlikar - Completed Use case diagram, Objectives, Slides and Presentation Recording, Conclusion and Evaluation

Justin Castro - I worked on the conclusion, evaluation, and architecture design slide.

Oluwawemimo Jayeoba - I completed the report formatting and outline creation, proofreading, the Comparison report, worked on the sequence diagram & comparison on slides, and refined the requirements following feedback received.

Albert Jimenez - I worked on the project scheduling, effort estimation, project timeline, and functional requirement slides.

Amrr Okasheh - Completed Test Plan for the Software, JUnit tests, Slides + Recording

Anish Patel - I worked on the project cost and pricing estimation, the staffing plan and project duration and the relevant slides.

3. DELIVERABLE #1 CONTENTS

3.1 Deliverable #1 Team and Task Delegation

Team Member	Completed Delegated Task ✓
Suhani Rana	Github ReadMe, Introduction, Feedback, Requirements
Sanyukta Bhadlikar	Use Case Diagram
Justin Castro	Architectural Design
Oluwawemimo Jayeoba	Report, Project Scope Github Commit & Sequence Diagrams
Albert Jimenez	Requirements, Project Scope
Amrr Okasheh	Github creation + adding members & TAs + Software Process Model
Anish Patel	Class Diagram

3.2 Initial Assumptions made for Deliverable #1

3.2.1 Assumptions made regarding the **requirements**:

- There is an account system with email/password sign-up, login, and logout.
- Live location sharing is opt-in.
- Users can belong to multiple trips at once and can see a list of them.
- Push notifications are supported and can be turned off or on.

3.2.2 Assumptions made regarding the **sequence diagrams**:

- User registration succeeds whenever credentials are entered correctly.
- Email service successfully sends emails whenever requested.
- The system follows a client-server architecture where the ClientApp represents the front-end interface and communicates

with back-end Service components (e.g., AuthService, TripService, BudgetService).

- Synchronous Communication
- External systems like Google Maps API or Push Notification API are treated as reliable third-party services that respond successfully to valid requests.
- Network delays, errors, or timeouts are not depicted for simplicity.
- The ClientApp provides immediate feedback to the user after receiving a response.
- The SystemScheduler for reminders and notifications is assumed to trigger events automatically at predefined intervals without user intervention.
- Users are assumed to be authenticated before performing protected actions (e.g., creating trips, adding expenses).

3.2.3 Assumptions made regarding the **US government compliance laws** for the app to operate successfully:

- The app fully complies with all applicable U.S. privacy and data protection laws.
- All user data is securely stored and transmitted using encryption
- The app does not infringe on copyrights, trademarks, or patents.
- The app is not directed toward children under 13.
- All third-party APIs (e.g., for maps, flight data, weather, and hotels) are used in accordance with their respective terms of service.
- The app requests and uses location data only after obtaining user consent.

3.3 Software Process Model (Incremental)

The software process model that we chose to implement is the Incremental Process Model. This approach allows us to build Voya in smaller, more manageable parts, enabling us to deliver a working system early while still maintaining flexibility. Voya has many features that are independent of one another, which demonstrates the strength of the incremental model by combining linear and parallel process flows and crafting each module in its own stage. This ensures steady progress by allowing

us to make adjustments whenever needed while continuing to add new features in future releases. Early feedback will also help us identify user pain points, leading to a better overall user experience in future iterations. The flexibility, modular structure, and rapid delivery make the incremental approach a great choice for Voya.

3.4 Feedback Received

3.4.1 Proposal Feedback

The team received positive feedback on the proposal. We worked collaboratively to expand the initial idea while refining the scope and addressing potential bottlenecks identified during the review.

3.4.2 Deliverable #1 Feedback

Feedback emphasized the need for each requirement to be atomic, representing a single, testable behavior, and to follow proper formatting for clarity and consistency. Based on this feedback, we revised both the functional and non-functional requirements to ensure they meet these standards.

3.5 Functional Requirements

3.5.1 User Authentication

3.5.1.1 The system shall allow a user to sign in using an email.

3.5.1.2 The system shall allow a user to sign in using a password.

3.5.1.3 The system shall validate the user's email and password combination

3.5.2 Trip Management (Admin Only)

3.5.2.1 The system shall allow an administrator to create a trip.

3.5.2.2 The system shall allow an administrator to edit trip details.

3.5.2.3 The system shall allow an administrator to delete a trip.

3.5.2.4 The system shall allow an administrator to invite members to a trip.

3.5.2.5 The system shall allow an administrator to remove members from a trip.

3.5.3 User Trip Access

3.5.3.1 The system shall allow a signed-in user to view trips they own.

3.5.3.2 The system shall allow a signed-in user to view trips they are a member of.

3.5.3.3 The system shall display trip titles for all trips visible to the user.

3.5.3.4 The system shall display trip dates for all trips visible to the

user.

3.5.4 Location Tracking

3.5.4.1 The system shall show the user's current location on a map during a trip.

3.5.4.2 The system shall show the last known location of other trip members.

3.5.5 Trip Budgeting

3.5.5.1 The system shall allow a user to add a budget entry for a trip.

3.5.5.2 The system shall allow a user to delete a budget entry for a trip.

3.5.5.3 The system shall calculate the total budget for a trip.

3.5.5.4 The system shall display the remaining budget for a trip.

3.5.6 Polling

3.5.6.1 The system shall allow a trip member to create a poll.

3.5.6.2 The system shall allow a trip member to vote in a poll.

3.5.6.3 The system shall allow a trip member to view poll results.

3.5.7 Notifications

3.5.7.1 The system shall send a notification when a user arrives at the trip destination.

3.5.7.2 The system shall send a notification when poll results are finalized.

3.5.7.3 The system shall send a notification before a deadline related to the trip.

3.6 Non-functional Requirements

3.6.1 Usability

3.6.1.1 The system shall provide onboarding screens for new users.

3.6.1.2 The system shall provide in-app tips to guide users.

3.6.1.3 The system shall allow first-time users to navigate core features without external help.

3.6.2 Performance

- 3.6.2.1 The system shall load trip data within 3 seconds of the request.
- 3.6.2.2 The system shall load maps within 3 seconds of the request.
- 3.6.2.3 The system shall deliver notifications within 5 seconds of the triggering event.
- 3.6.2.4 The system shall support 500 concurrent users without performance degradation.

3.6.3 Space

- 3.6.3.1 The mobile app shall not exceed 200 MB on the user's device.
- 3.6.3.2 The system shall limit server-stored data to 10 MB per trip.

3.6.4 Dependability

- 3.6.4.1 The system shall display error messages when a network failure occurs.
- 3.6.4.2 The system shall display error messages for invalid user inputs.
- 3.6.4.3 The system shall back up all critical trip data.
- 3.6.4.4 The system shall back up all budget data.

3.6.5 Security

- 3.6.5.1 The system shall use secure authentication methods.
- 3.6.5.2 The system shall encrypt all communication between client and server.
- 3.6.5.3 The system shall restrict access to trip details to authorized users only.

3.6.6 Environmental

- 3.6.6.1 The system shall operate on iOS devices.
- 3.6.6.2 The system shall operate on Android devices.
- 3.6.6.3 The system shall support light mode.
- 3.6.6.4 The system shall support dark mode.

3.6.7 Operational

- 3.6.7.1 The system shall be operational 24/7.
- 3.6.7.2 The system shall conduct maintenance during low-traffic hours.
- 3.6.7.3 The system shall support common mobile screen sizes.

3.6.8 Developmental

3.6.8.1 The system shall be developed using industry-standard technologies.

3.6.8.2 The system shall use a version-controlled GitHub repository.

3.6.8.3 All team members shall use the shared repository for collaboration.

3.6.9 Regulatory

3.6.9.1 The system shall allow users to delete their accounts.

3.6.9.2 The system shall delete all personal data when a user deletes their account.

3.6.10 Ethical

3.6.10.1 The system shall not collect user data without consent.

3.6.10.2 The system shall not share user data without consent.

3.6.10.3 The system shall disclose how user data is used.

3.6.10.4 The system shall disclose how user data is stored.

3.6.11 Accounting

3.6.11.1 The system shall track all trip expenses.

3.6.11.2 The system shall track all trip budgets.

3.6.11.3 The system shall calculate remaining funds.

3.6.11.4 The system shall timestamp all budget entries.

3.6.11.5 The system shall record the user who created each budget entry.

3.6.12 Safety & Security

3.6.12.1 The system shall not disclose a user's location to external parties without explicit consent.

3.7 Use Case Diagram Overview

Purpose:

The purpose of this use case diagram is to show the major interactions between the users, the trip administrator, and external and notification systems with the main Voya app. This diagram visually shows how different actors engage with the system to perform tasks such as creating a trip, budgeting for the trip, deciding on activities, and interacting with a map. The diagram also shows other features that are inclusions or extensions of the core tasks a user can perform in the app, such as notifications for bookings and voting.

Flow Summary:

- **User**
 - A user can log in, create a trip activity and vote on it, manage the budget, and view the trip's dashboard.
 - Users can look at the voting results, budget summaries, and maps of the trip through the inclusions/extensions of the core functionalities.
 - The "Share Live Location" feature is an extension of the "View Map" function that allows users to see where others are during the trip itself.
- **Admin**
 - Administrators are the only ones who can create, manage, and/or delete trips. They are also the only ones who can invite or remove members.
- **System interactions**
 - The notification system communicates with the app to send alerts to the users, such as updates on the trip, voting results, and more.
 - External systems (such as Google Maps and Mail) are integrated into the map function for easy visualization.

Diagram:

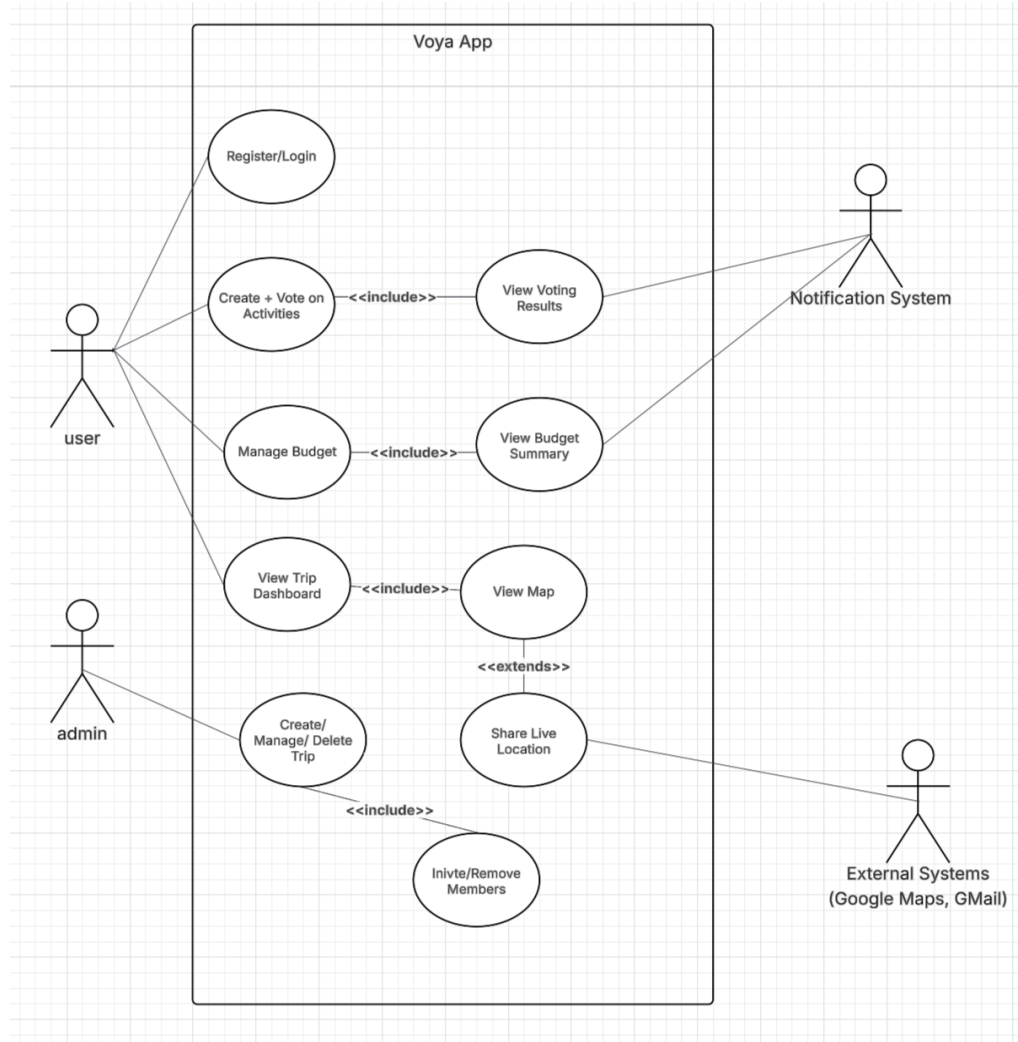


Figure 3.7 - Use Case Diagram

3.8 Sequence Diagram Overview

3.8.1 Sequence Diagram – User Registration and Login

Purpose:

This sequence diagram shows how users register and log in to the Voya system. It illustrates the interaction between the user, the client application, the authentication service, and the user database.

Flow Summary:

- The user enters registration info, which the ClientApp sends to AuthService.
- AuthService stores the user credentials in the database.
- The user logs in; the system verifies credentials and displays the dashboard upon success.

Diagram:

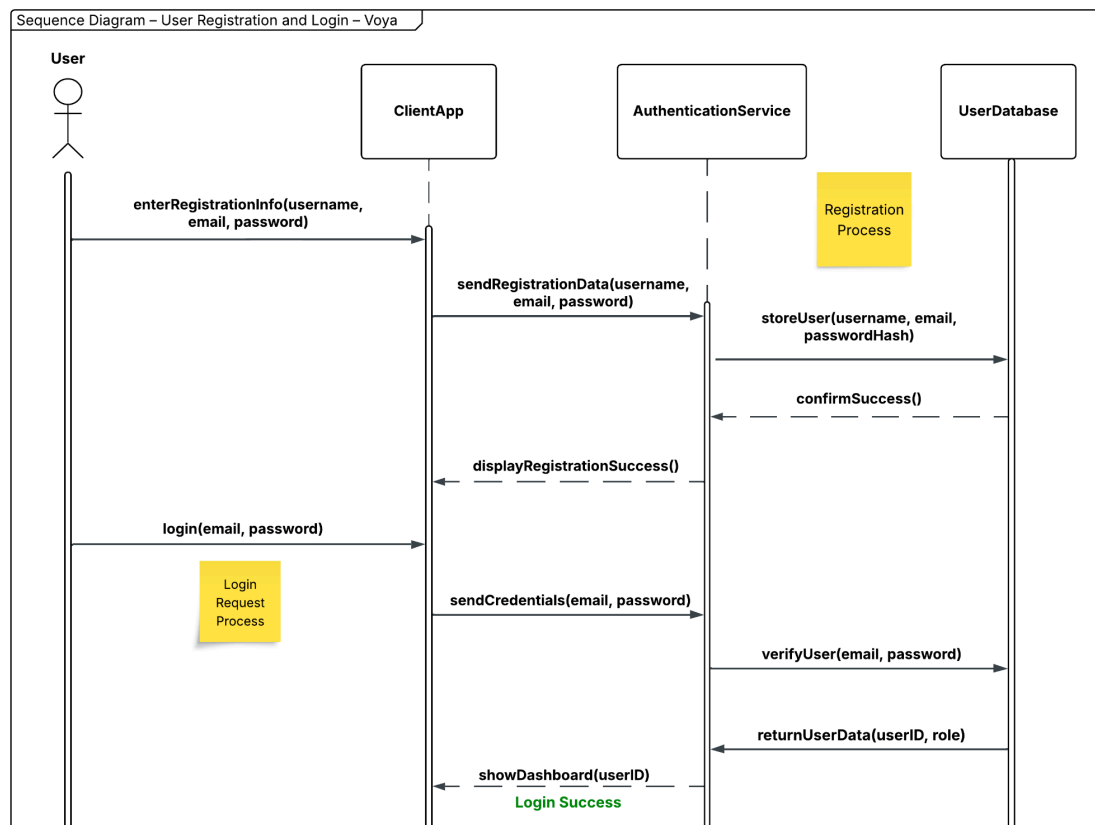


Figure 3.8.1 – Sequence Diagram for User Registration and Login

3.8.2 Sequence Diagram – Create Trip

Purpose:

This diagram depicts how a user creates a trip. It involves the client application sending trip details to the TripService, which validates and stores the data in the database.

Flow Summary:

- The user clicks “Create Trip.”
- The system validates and stores trip details in TripDatabase.
- A confirmation message is displayed to the user.

Diagram:

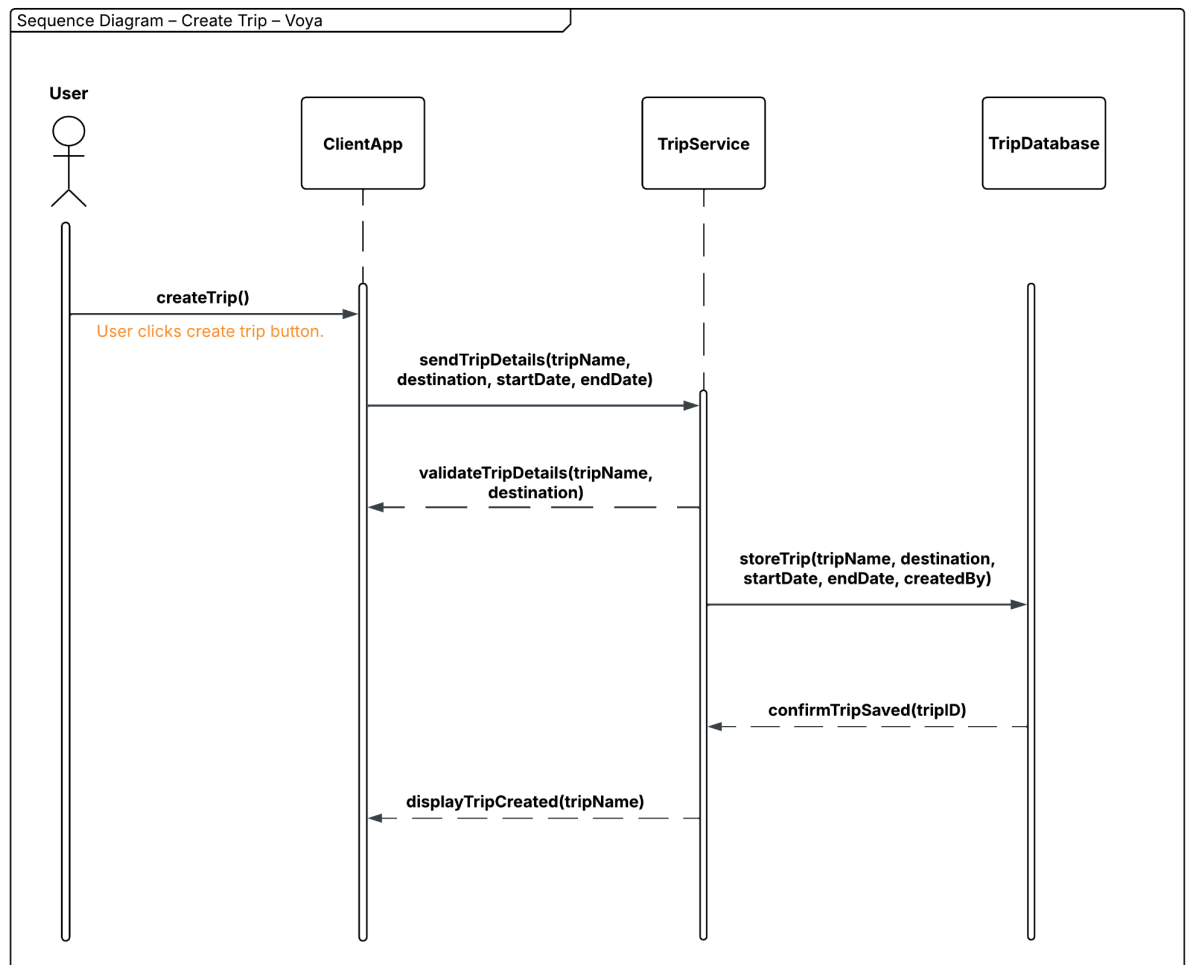


Figure 3.8.2 – Sequence Diagram for Create Trip

3.8.3 Sequence Diagram – Invite Members

Purpose:

Shows how a trip owner invites members to a trip and how the system verifies and sends invitation emails.

Flow Summary:

- The trip owner enters emails.
- The system checks for valid users and sends invitations via EmailService.
- Client displays “Invites sent successfully.”

Diagram:

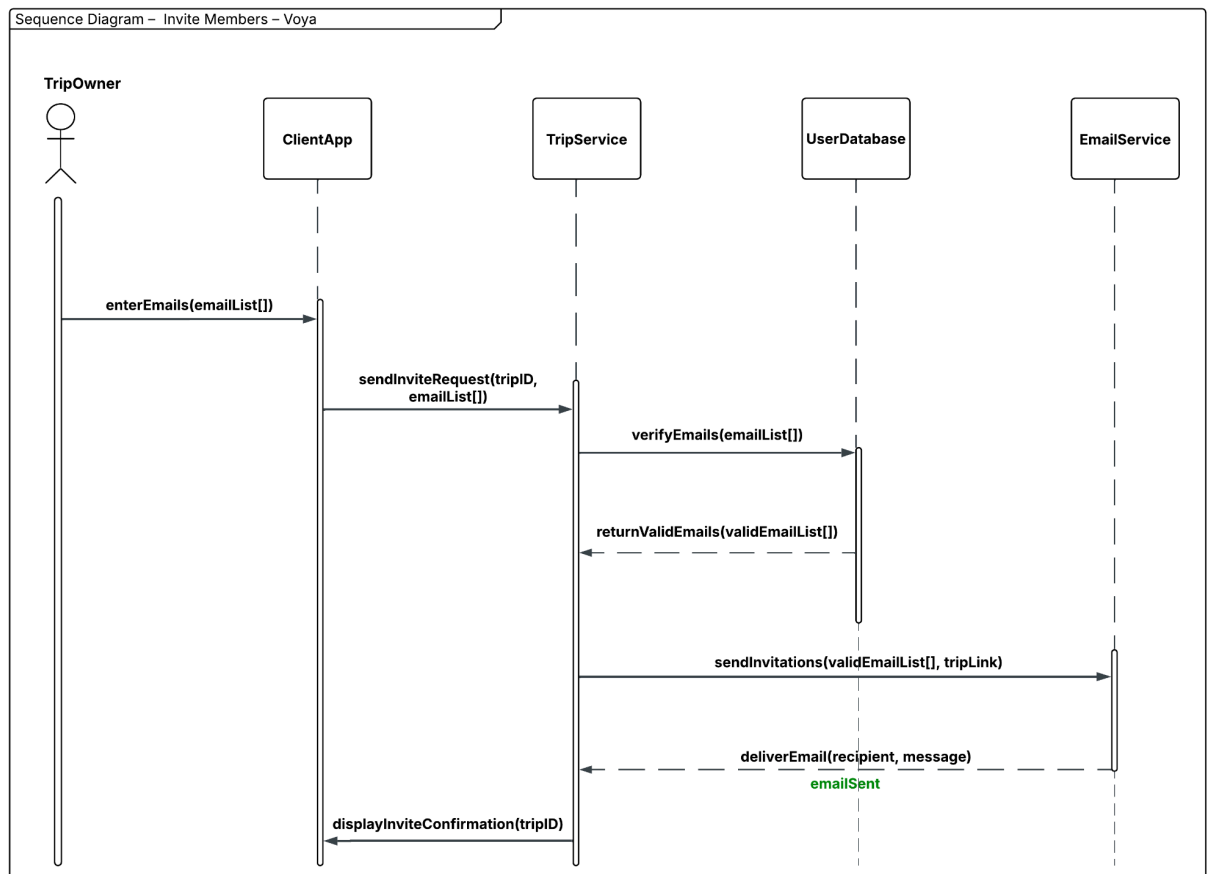


Figure 3.8.3 – Sequence Diagram for Invite Members

3.8.4 Sequence Diagram – Add and Vote on Activities

Purpose:

This diagram represents how members can propose new activities and vote on them collaboratively.

Flow Summary:

- Member adds an activity → ActivityService saves it.
- Members vote → the service updates the vote count in TripDatabase.
- Results refresh on all members' screens.

Diagram:

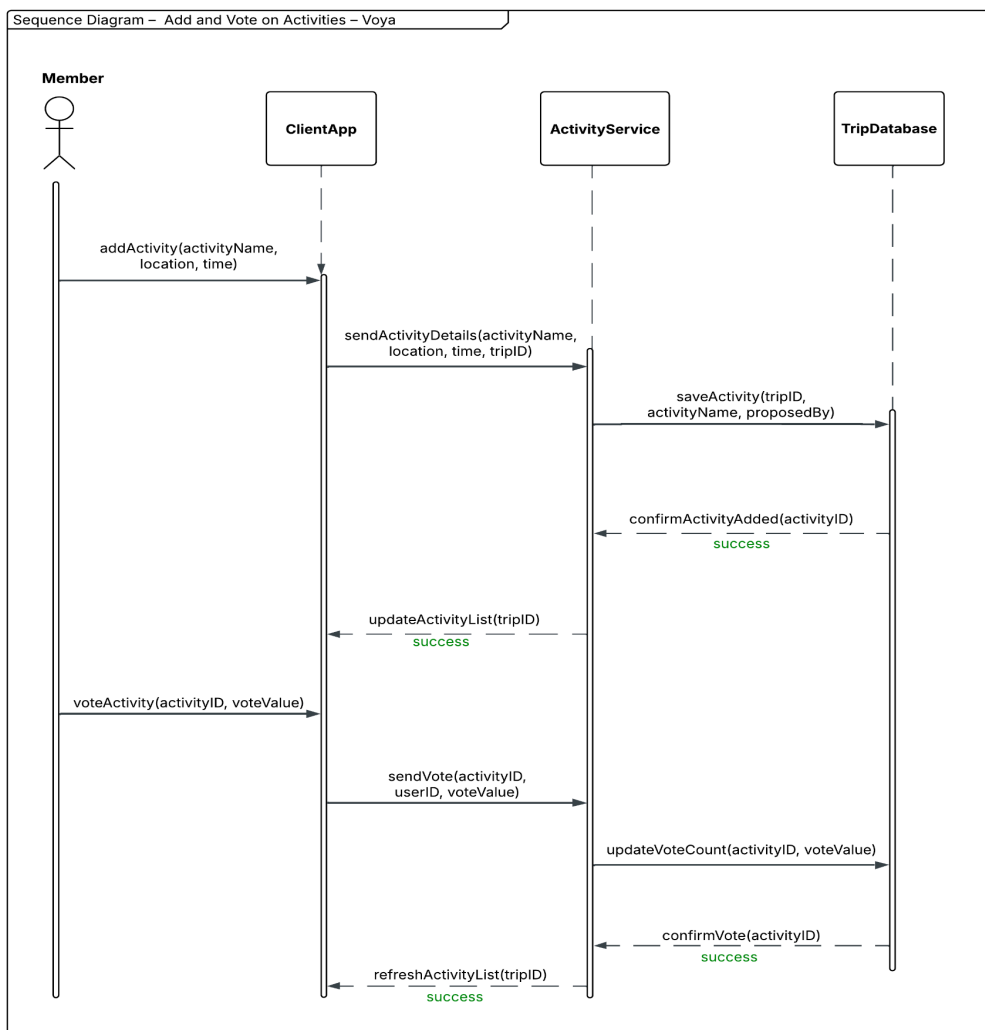


Figure 3.8.4 – Sequence Diagram for Add and Vote on Activities

3.8.5 Sequence Diagram – Manage Budget

Purpose:

Depicts how users record expenses and view budget summaries for each specific trip.

Flow Summary:

- The user adds expense details.
- BudgetService stores data and updates totals.
- User views a summary of total spent and per-member share.

Diagram:

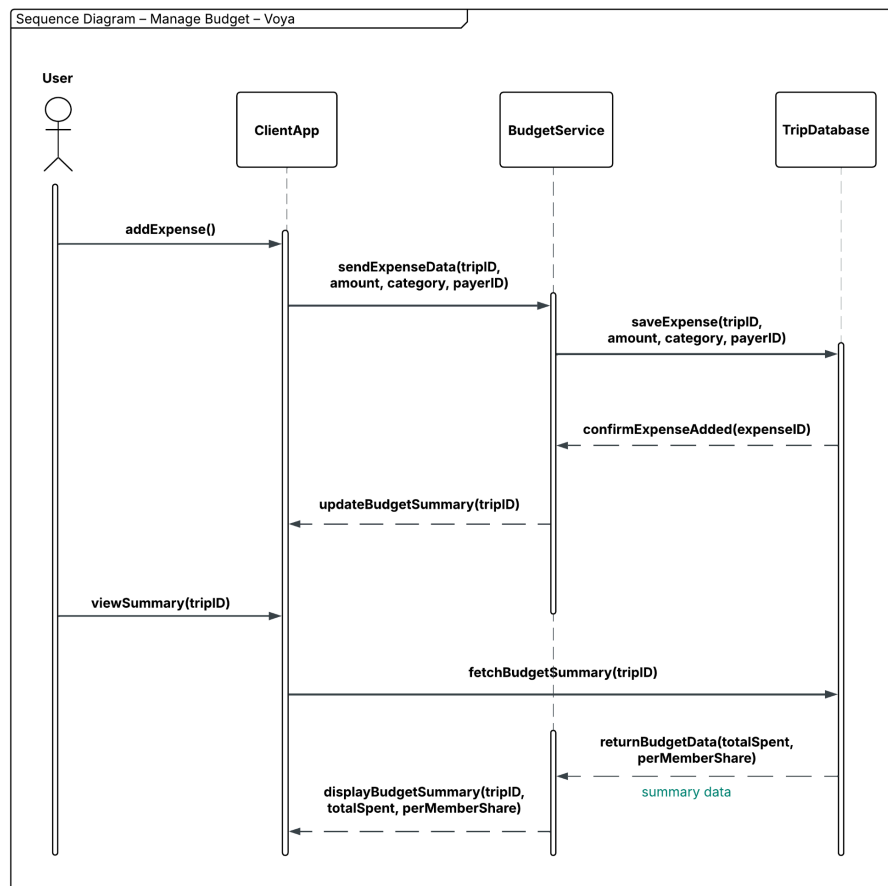


Figure 3.8.5 – Sequence Diagram for Manage Budget

3.8.6 Sequence Diagram – View Itinerary and Map

Purpose:

Shows how users request their trip itinerary and route information with Google Maps integration.

Flow Summary:

- The user requests an itinerary.
- The system retrieves the itinerary from the DB and calls the Google Maps API.
- The map and itinerary are displayed in the app.

Diagram:

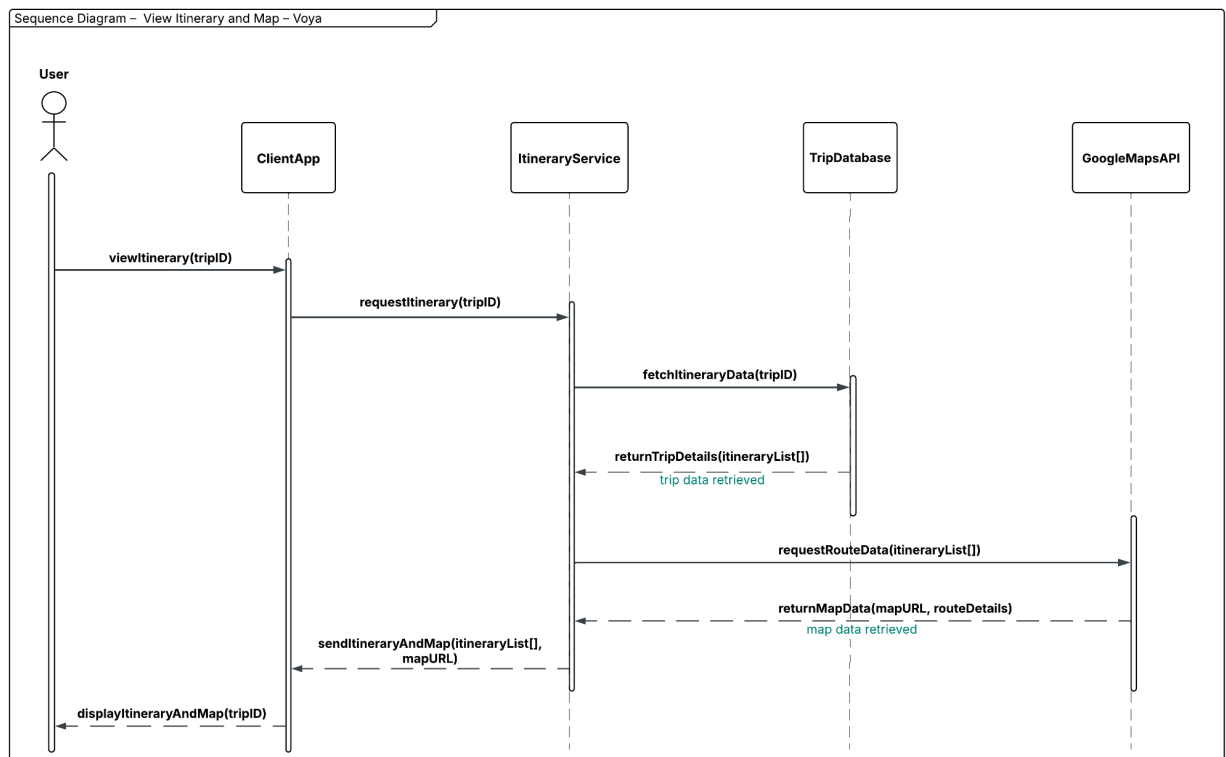


Figure 3.8.6 – Sequence Diagram for View Itinerary and Map

3.8.7 Sequence Diagram – Notifications and Reminders

Purpose:

Describes how the system automatically sends reminders and notifications to users.

Flow Summary:

- Scheduler triggers a notification event.
- NotificationService checks upcoming trips and sends push notifications via API.
- The user receives and opens a notification.

Diagram:

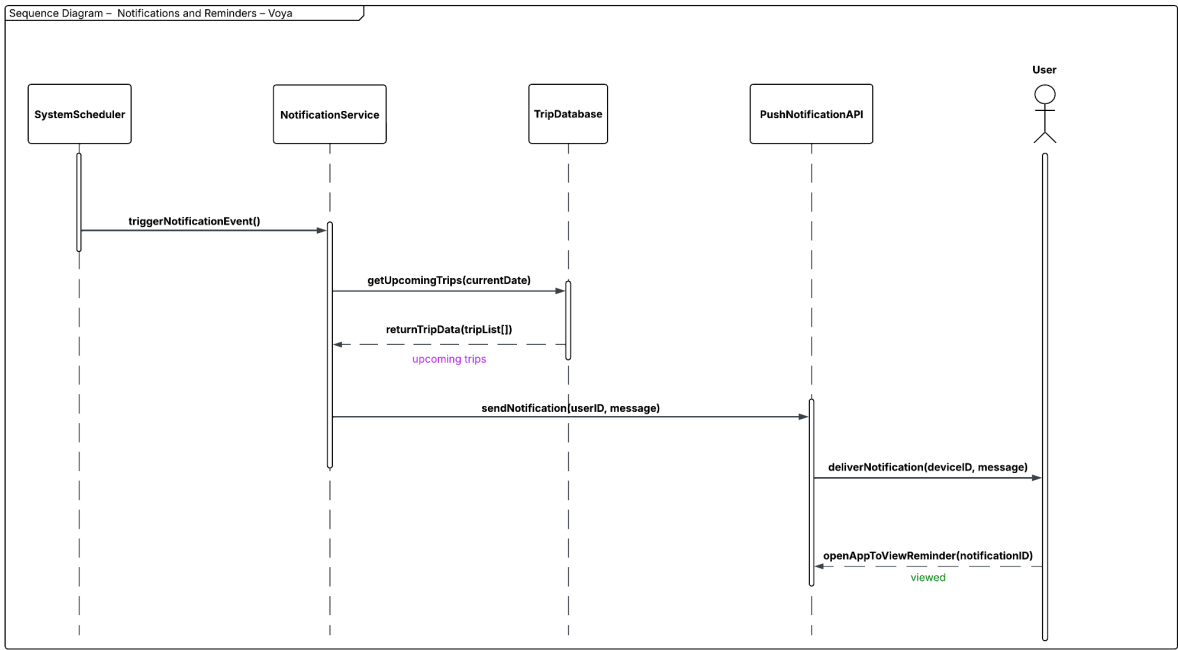


Figure 3.8.7 – Sequence Diagram for Notifications and Reminders

3.9 Class Diagram Overview

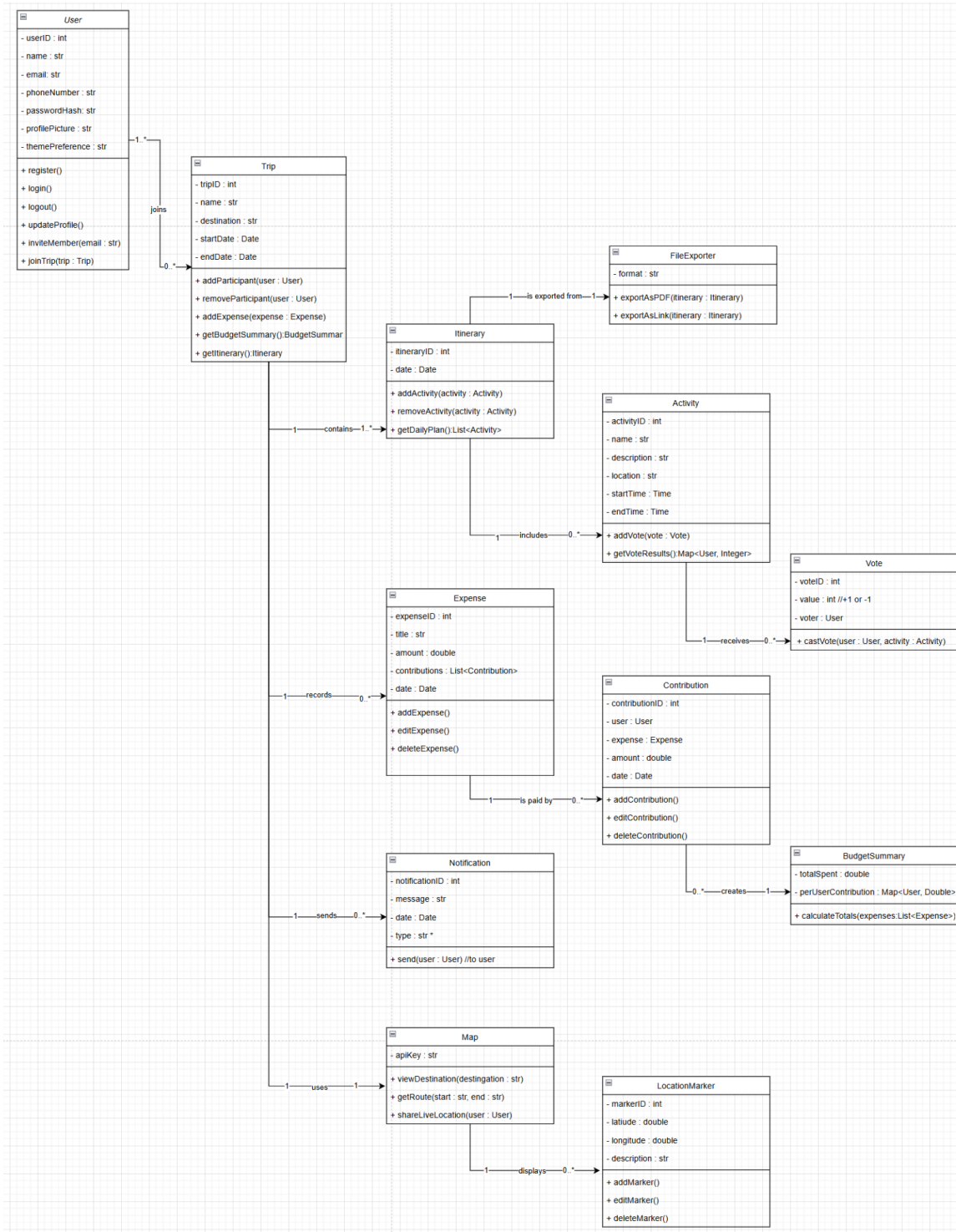


Figure 3.9 – Class Diagram

3.10 Architectural Design Recap

Voya will utilize a client-server architectural design as it provides an efficient way for multiple users to access travel information, communicate with one another, and modify information regardless of where they are located. It will also provide an efficient way to store data, information, and access external APIs for Google Maps integration and notifications.

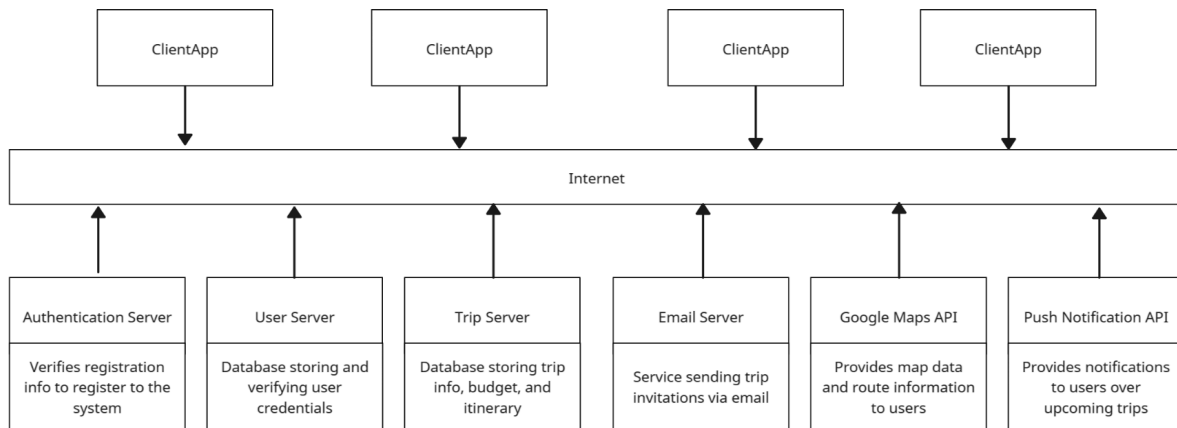


Figure 3.10 – Client-Server Architecture Design

4. Project Scheduling, Cost, Effort, and Pricing Estimation

4.1 Project Scheduling

4.1.1 Estimated Start and End Dates

Dates: The project implementation will begin after the Fall 2025 is over to allow planning to be completed.

- Estimated Start Date: January 20, 2026
- Estimated End Date: April 17, 2026

Reasoning: As we see later, during the calculation of effort estimates, only 3.4 person months of pure development effort. We will extend this time to allow:

- The team being full of college students with daily limited availability
- Testing across multiple devices
- Anything that might happen for contingency

4.1.2 Working Hours, Weekends, and Schedule Assumptions

Assumptions: Assumptions regarding the schedule are mainly derived from deliverable #1 assumptions and some added ones, such as:

- Reliable third-party services such as Google Maps, API, email service, etc.
- Automated triggers for push notifications.
- Immediate feedback to the user after service responses.
- The project members will be the ones developing the app.
- The project members will be attending full-time school during the project development phase.
- Client Server Architecture: The ClientApp interacts with 3GL backend services: AuthService, TripService, and BudgetService.

Working Hours: Working hours will align with the project members' class schedules and academic responsibilities.

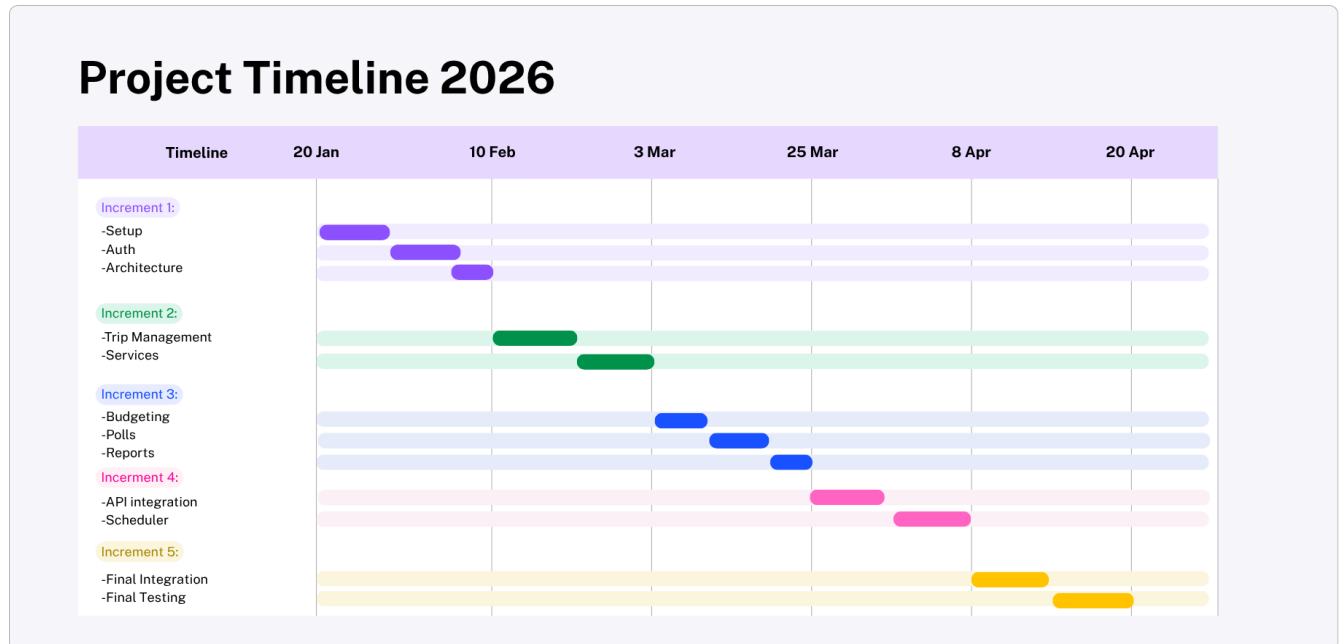
Working Hours per Day: There are 7 members, with each member contributing around 2 hours per weekday at their own discretion.

- 14 person-hours/day.
- 70 person-hours/week.

Weekends:

- Weekends will not be counted as planned working days.
- Weekends will be used as contingency in case some unforeseen events occur that delay the schedule.

4.1.3 Project Timeline (Gantt Chart)



4.2 Effort Estimation

4.2.1 Chosen Estimation Technique (Application Composition)

Application Composition Model: This model will be selected as the team's algorithmic estimation method due to the model aligning more with what Voya consists of:

- Screens / UI views
- Report summaries like the budget and poll results.
- Reusable third-party services like maps, notifications, and API.

4.2.2 Rationale and Calculation Steps

Rationale: As stated earlier, Voya is primarily made of screens and a report-style attribute that aligns with Object Points in the model. The system also relies on third-party apps, making reusability relevant. This approach is simpler, relevant, and more precise for the type of application we are developing compared to Function Point Analysis.

Calculation: During this, we will be going through the steps for the Application Composition Model.

- **Step One: Count the Number of screens, reports, and 3GL components**
 - Screens
 1. Login / Sign-up
 2. Dashboard
 3. Trip Details
 4. Budget Screen
 5. Polls/Voting Screen
 6. Notifications/Activity Feed
 - Reports
 1. Budget Summary Report
 2. Poll Result Report
 - 3GL Components: Backend components written in a 3GL language
 1. AuthService
 2. TripService
 3. BudgetService

This adds up to: 6 screens, 2 reports, and 3 medium 3GL components.

- **Step Two: Determine Complexity Levels**
 - Screens are Simple
 - Reports are Complex
 - Backend service modules are medium 3GL components
- **Step Three: Add the Weighted Counts**
 - Simple Screen: 1 OP
 - Complex Report: 8 OP
 - Medium 3GL Component: 10 OP
- **Step Four: Compute Object Points**
 - Screens: $6 * 1 \text{ OP} = 6 \text{ OP}$
 - Reports: $2 * 8 \text{ OP} = 16 \text{ OP}$
 - 3GL Components: $3 * 10 \text{ OP} = 30 \text{ OP}$
 - Total OP: $6 + 16 + 30 = 52 \text{ OP}$

- **Step Five: Estimate Reuse and Compute New Object Points**
 - We assumed the use of Google Maps API, email provider, and push notification API
 - All external systems are trusted third parties
 - Immediate feedback occurs after responses and schedules trigger notifications automatically
 - Reuse can then be said to be around 35%

$$\text{NOP} = \text{OP} * (100 - \text{Reuse})/100$$

$$= 52 * 0.65 = 33.8 \text{ or rounded up to } \mathbf{34 \text{ NOP}}$$
- **Step Six: Determine Productivity**
 - Low domain experience due to student developers
 - Average development tools used
 - Using a 3GL for backend services
 - Due to the average of low developer experience and capability, and nominal ICASE maturity and capability, as stated in the slides

$$\text{PROD} = (7 + 13) / 2 = 10$$

$$\text{PROD} = 10 \text{ OP / Person-month}$$
- **Step Seven: Compute the person-month Effort**
 - Effort PM = NOP / PROD
 - **Effort = 34 / 10 = 3.4 person-months**
 - Converted to hours is
 - 3.4 person-months * 160 hours (4 40-hour weeks) = **544 person-hours**

4.3 Cost and Pricing Estimation

4.3.1 Hardware Cost Estimates

Although Voya is primarily a software-based system, several hardware resources are needed for development and testing. These include devices, cloud servers, and additional development equipment. Devices will be selected based on the most efficient cost-to-effectiveness ratio, as high-end devices will not be needed for testing.

Developer Testing Devices:

- Android test devices (mid-range): \$250-\$350
- iPhone test devices (older models): \$350-\$450

- Estimated total: \$600-\$800

Server Hosting / Cloud Services

- Cloud backend server (AWS/Azure) testing instance for 3 months: \$30-\$50 per month
- Database hosting (managed PostgreSQL): \$10-\$20 per month
- Cloud storage for images, receipts, and trip data: \$5-\$10 per month
- Estimated total: \$150 for the given development period

Backup and Network Hardware

- External SSD for backup: \$100
- Estimated total: \$100

Total estimated hardware costs: \$850-\$1050

4.3.2 Software Cost Estimates

Development Tools:

- Android Studio - Free
- Xcode - Free
- VSCode / IntelliJ - Free

Version Control:

- GitHub Free Plan - Free

APIs (Estimated Usage-Based Costs):

- Google Maps API: \$5-\$20 depending on load during testing
- Email API: Free tier available
- Push Notifications Firebase Cloud Messaging: Free

Testing Tools:

- Emulator tools - Generally free
- Potential cloud device testing - \$20-\$30

Total Estimated Software Cost: \$30-\$50 for the given development period

4.3.3 Personnel and Training Cost Estimates

Personnel will make up the most significant portion of most projects when it comes to salaried employees. As this is a student-led project, the cost for personnel will be close to 0, as most work will be done

voluntarily. Below is a symbolic rate for academic work if hiring students were needed.

Assumptions:

- All seven team members contribute based on availability, approximately 2 hours per weekday.
- Work is completed voluntarily
- No formal salaries are paid

Symbolic Rate for Academic Work:

- Calculating both realistic and minimal cost values, we can use a symbolic hourly rate that would be typical for an inexperienced student.
- \$10 per hour for the student developer rate
- The rate is low and reflects: lack of professional experience, academic context, and the goal of the project is skill building, not income.

Personnel Effort Cost:

- From the previous section of the Application Composition Model, the estimated time worked is 544 person-hours * \$10 per hour.
- Total estimated personnel cost: \$5400

Equivalent Industry Personnel Cost:

- If Voya were developed professionally, the low-end range for hourly rate would be about \$25-\$35 per hour
- Total estimated Industry personnel cost: \$13600-\$19040

Training Cost:

- As the team would already have foundational knowledge and most additional training needed to work with APIs and cloud-based services are free, the costs would be low if utilizing free trials.
- Estimated Training Cost: \$0-\$30

4.3.4 Total Cost and Pricing Strategy

Total Development Cost Estimate:

- Hardware: \$850-\$1050
- Software: \$30-\$50

- Personnel: \$5400
- Training: \$0-\$30
- Total Estimated Cost: \$6280-\$6530

Pricing Strategy:

- To position Voya competitively in the market, a freemium model would be desirable.
- Free Version: basic itinerary, and group trip view
- Premium tier(\$2.99 per month): Shared budgets, polling features, live location, unlimited trips
- This model balances revenue generation with accessibility, as this is primarily targeted towards students with less income.

4.4 Staffing Plan and Project Duration

4.4.1 Staffing Plan

Because the development team is composed of students with varying strengths and the staffing plan will be designed to distribute responsibility evenly while allowing for proper overlap for support.

The table below assigns roles to the students, but keep in mind that all members should be able to pivot to help different members.

Role	Responsibilities	Assigned Members
Project Coordinator	Organizes meetings and manages the timeline	1 student
Front-End Developer	UI, navigation, screens, and various interfaces	1 / 2 Students
Back-End Developer	AuthService, TripService, BudgetService, and API endpoints	1 / 2 Students
Database Engineer	Schema design and overall database	1 Student

	construction	
Quality Assurance	Unit testing and bug reporting	1 Student
API Integration Dev	Organizes the API usage and integration	1 Student

Creation of Voya will be based on the Gantt Chart timeline; as such, many of the students' roles will not be able to start upon the creation of the project, meaning they will be free to help out in other aspects of the project.

4.4.2 Development Duration

Below is an estimated schedule consistent with the Jan 20 - Apr 17 schedule:

1. Requirement Finalization: 1 Week
2. UI/UX Prototyping: 1 Week
3. Back End Core Development: 3 Weeks
4. API & Third Party Integration: 1 Week
5. Front End Integration: 2 Weeks
6. System Testing and Bug Fixing: 2 Weeks
7. Final Polishing and Deployment: 1 Week

5. Test Plan for the Software

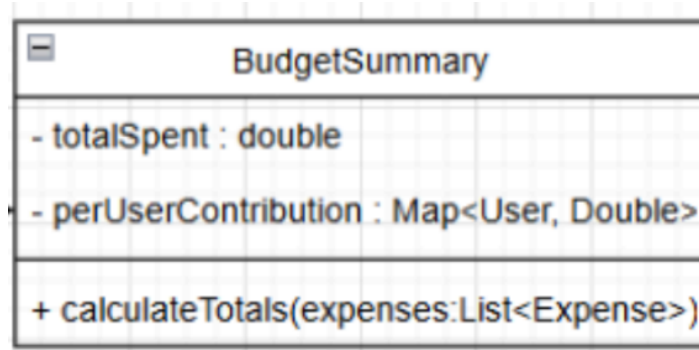
5.1 Overview of Test Strategy

For our project Voya, we implemented black-box testing on our budget calculation logic in order to see if it behaves correctly given some trip data. We automated the unit tests with JUnit, focusing on the BudgetSummary class that relies on the Contribution and Expense classes as well. We chose this class because it implements core Voya functionality, and its correctness is critical for a good user experience. The test focused on:

- Correct summation of the total trip spending from multiple Expense objects (flight, hotel, Uber, food, etc.)

- Correct summation of individual user contributions from Contribution objects (how much a user contributed to the Expenses)
- Correct internal data structure, making sure that the map of user and contribution is correctly connected as expected.

5.2 Unit Selected for Testing (Description and Rationale)



The unit I selected was BudgetSummary. BudgetSummary is a class that has two variables (totalSpent, perUserContribution), where totalSpent is a double that holds the value of the total amount of money spent on each expense, while perUserContribution is a data structure that holds each user's total contribution amount, based on the contribution objects associated with each expense. We chose this class to test because it serves as a core and integral function of travel planning, as it allows users to understand how much each person is contributing to the expenses that were listed for the trip.

5.3 Test Case Design and Expected Outcomes

The test case design involved creating some typical expenses a user might experience with their group when going on a trip. Each Contribution represented a group member and their specific dollar value contributed towards an expense, while each Expense object represented a common trip cost, such as food, transportation, and hotel fees. Real names and specific expense names were intentionally used for clarity and presentation purposes. These will be changed to more generalized names in the final product. The test

verified that when three expenses that totaled \$2800 were passed into the `calculateTotals` method, the correct output was computed. It then confirmed that all seven contributors were properly represented in the `perUserContribution` map by checking its size. Finally, the individual entries in the map were validated to ensure that each member's total contribution accurately matched the amount they provided.

5.4 Unit Implementation Code (Sample Method / Class)

```
1  import java.util.List;
2  import java.util.HashMap;
3  import java.util.Map;
4
5
6  public class BudgetSummary { 3 usages
7
8      double totalSpent; 2 usages
9      Map<String, Double> perUserContribution; 2 usages
10
11      public BudgetSummary(double totalSpent, Map<String, Double> perUserContribution) 2 usages
12      {
13          this.totalSpent = totalSpent;
14          this.perUserContribution = perUserContribution;
15      }
16
17
18
19      public double getTotalSpent() { no usages
20          return totalSpent;
21      }
22      public Map<String, Double> getPerUserContribution() { no usages
23          return perUserContribution;
24      }
25
26  @
27      public static BudgetSummary calculateTotals(List<Expense> expenses) no usages
28      {
29          double total = 0.0;
30          Map<String, Double> contributionsMap = new HashMap<>();
```

```

30
31     // if we dont have any expenses in the budgetSummary default to 0 value for total spent
32     if(expenses == null || expenses.isEmpty())
33     {
34         return new BudgetSummary(totalSpent: 0, contributionsMap);
35     }
36
37     // if we have contributions then we have to loop through the expenses
38
39     for (int i = 0; i < expenses.size(); i++)
40     {
41         Expense e = expenses.get(i);
42         total += e.getAmount();
43
44         List<Contribution> contribs = e.getContributions();
45         for( int j = 0; j < contribs.size(); j++)
46         {
47             Contribution c = contribs.get(j);
48
49             contributionsMap.merge(c.getUserID(), c.getAmount(), Double::sum);
50
51         }
52     }
53
54     return new BudgetSummary(total, contributionsMap);
55
56 }
57
58 }
59

```

The above images show the unit that is being tested.

The images below show the additional classes that were needed to implement the BudgetSummary class.

```

BudgetSummary.java  BudgetSummaryTest.java  Contribution.java  Expense.java
1  public class Contribution { 2 usages
2      private String userID; 2 usages
3      private double amount; 2 usages
4
5      public Contribution(String userID, double amount) { no usages
6          this.userID = userID;
7          this.amount = amount;
8      }
9
10     public String getUserID() { 1 usage
11         return userID;
12     }
13
14     public double getAmount() { 1 usage
15         return amount;
16     }
17 }
18

```

```
BudgetSummary.java BudgetSummaryTest.java Contribution.java Expense.java ×
1 import java.util.Date;
2 import java.util.List;
3
4 public class Expense { 2 usages
5     private int expenseID; 1 usage
6     private double amount; 2 usages
7     private List<Contribution> contributions; 2 usages
8
9     public Expense(int expenseID, double amount, List<Contribution> contributions, Date date) { no usa
10         this.expenseID = expenseID;
11         this.amount = amount;
12         this.contributions = contributions;
13     }
14
15     public double getAmount() { 1 usage
16         return amount;
17     }
18
19     public List<Contribution> getContributions() { 1 usage
20         return contributions;
21     }
22 }
```

5.5 Automated Unit Testing (JUnit)

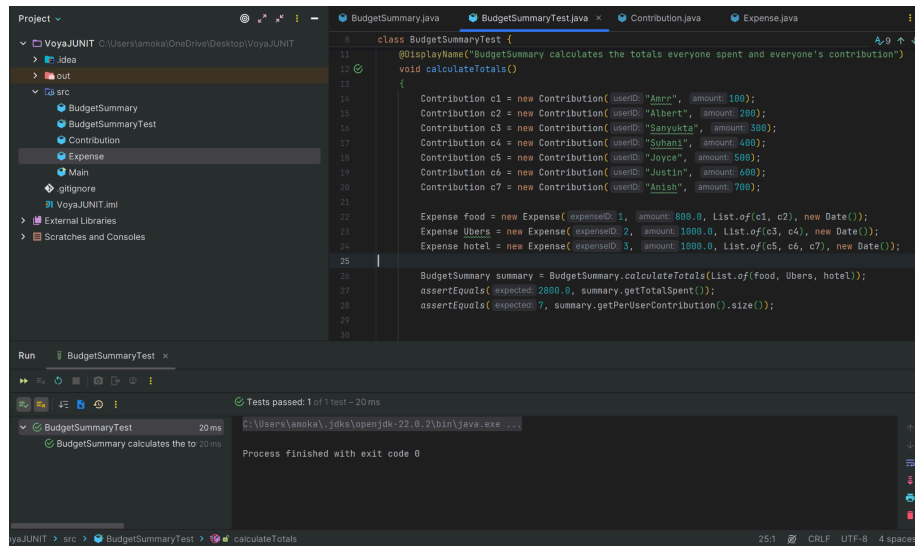
```
@Test
@DisplayName("BudgetSummary calculates the totals everyone spent and everyone's contribution")
void calculateTotals()
{
    Contribution c1 = new Contribution( userID: "Amrr", amount: 100);
    Contribution c2 = new Contribution( userID: "Albert", amount: 200);
    Contribution c3 = new Contribution( userID: "Sanyukta", amount: 300);
    Contribution c4 = new Contribution( userID: "Suhani", amount: 400);
    Contribution c5 = new Contribution( userID: "Joyce", amount: 500);
    Contribution c6 = new Contribution( userID: "Justin", amount: 600);
    Contribution c7 = new Contribution( userID: "Anish", amount: 700);

    Expense food = new Expense( expenseID: 1, amount: 800.0, List.of(c1, c2), new Date());
    Expense Ubers = new Expense( expenseID: 2, amount: 1000.0, List.of(c3, c4), new Date());
    Expense hotel = new Expense( expenseID: 3, amount: 1000.0, List.of(c5, c6, c7), new Date());

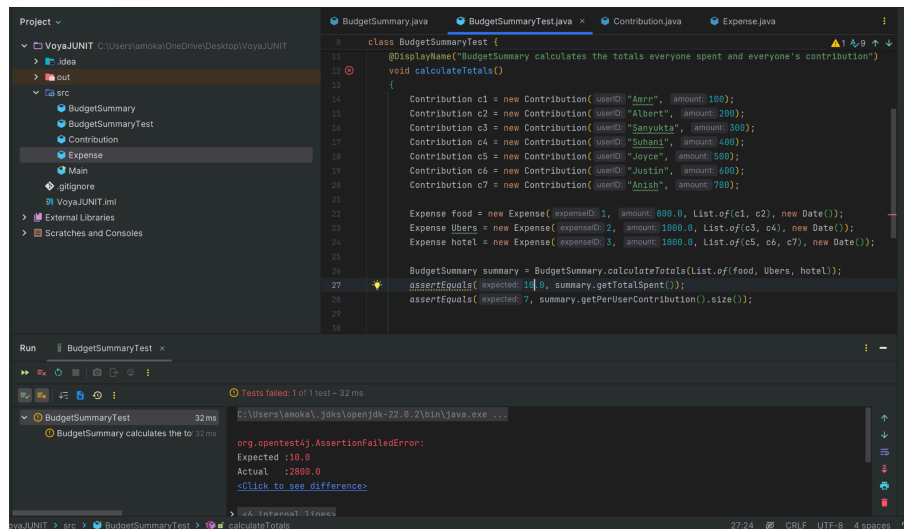
    BudgetSummary summary = BudgetSummary.calculateTotals(List.of(food, Ubers, hotel));
    assertEquals( expected: 2800.0, summary.getTotalSpent());
    assertEquals( expected: 7, summary.getPerUserContribution().size());

    assertEquals( expected: 100.0, summary.getPerUserContribution().get("Amrr"));
    assertEquals( expected: 200.0, summary.getPerUserContribution().get("Albert"));
    assertEquals( expected: 300.0, summary.getPerUserContribution().get("Sanyukta"));
    assertEquals( expected: 400.0, summary.getPerUserContribution().get("Suhani"));
    assertEquals( expected: 500.0, summary.getPerUserContribution().get("Joyce"));
    assertEquals( expected: 600.0, summary.getPerUserContribution().get("Justin"));
    assertEquals( expected: 700.0, summary.getPerUserContribution().get("Anish"));
}
```

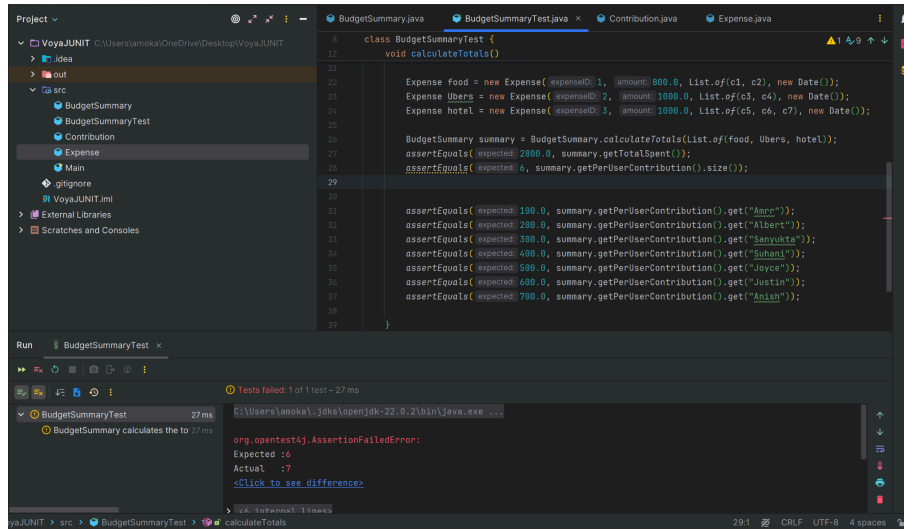
5.6 Test Results and Observations



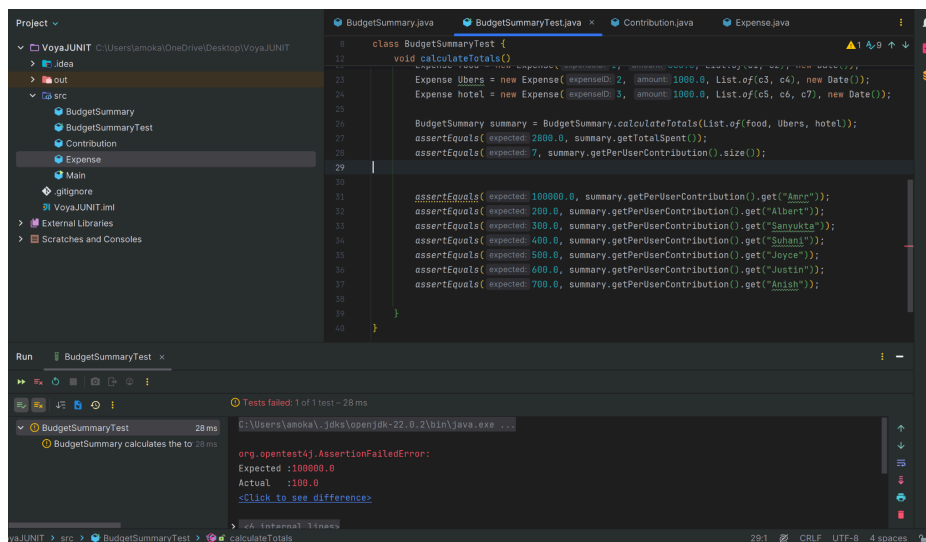
The image above shows a successful test case, in which all the values are correct and everything is running as the user expects.



The image above shows an incorrect test case when testing getTotalSpent, showing each person's contribution totaling to 2800, but it is expecting a value of 10.



The above image shows an incorrect test case, showing the size of the group is 7, while it's expecting an incorrect value of 6.



The above image shows an incorrect test case, showing that getPerUserContribution is expecting 10,000, but Amrr is only contributing 100.

In summary, the black-box testing confirms that the BudgetSummary class is working as intended, and calculateTotals gives the correct output.

6. Comparison with Similar Systems

6.1 Overview of Existing Travel Planning Platforms

There are various travel-planning applications available today, and to understand Voya's position within the broader travel-planning ecosystem, we evaluated several of the most widely used travel platforms. These platforms vary in purpose, functionality, collaboration capabilities, and target user groups. Below is an overview of four major platforms, listed roughly in order of popularity:

Tripadvisor

Tripadvisor offers an extensive database of reviews and recommendations for hotels, restaurants, attractions, and activities. Its strength lies in its powerful search, ranking, and suggestion features, making it one of the most widely used discovery tools for travelers. However, Tripadvisor focuses primarily on content and recommendations rather than collaboration. It lacks shared itinerary building, group budgeting, group communication, or activity polling, making it less suited for coordinated trip planning among multiple travelers [1].

Triplt

Triplt automatically generates itineraries by scanning travel confirmation emails from airlines, hotels, and rental services. It excels at personal organization by consolidating reservations into a clean, chronological itinerary. Despite its usefulness, Triplt is not designed for collaborative planning and does not include group features such as shared budgeting, real-time decision-making, or location sharing [2].

Wanderlog

Wanderlog provides users with tools to create shared itineraries, manage reservations, and plan trips through an interactive map-based interface. It is one of the few platforms that intentionally supports collaborative trip planning. However, Wanderlog still lacks more structured group functionalities, such as detailed budgeting and expense tracking, activity polling for democratic group decisions, and live location tracking during trips, which limits its ability to fully coordinate group travel [3].

Wego

Wego focuses primarily on price comparison and travel booking. It collects flights, hotels, and travel deals across multiple providers, making it highly effective for finding and booking affordable travel options. Despite its strong search and comparison capabilities, Wego does not provide tools for group itinerary building, shared budgeting, live coordination, or collaborative decision-making [4].

The Voya app essentially unifies and improves some key strengths from all four platforms into a single, cohesive system tailored for group travelers. Tripadvisor and Wego excel in discovery and booking, while Triplt and Wanderlog offer limited trip organization. Voya stands out as the only platform intentionally designed for real-time, collaborative group planning, integrating budgeting, live location sharing, activity voting, Google Maps interaction, and coordinated decision-making into one application.

6.2 Comparative Feature Table (Voya vs. Competitors)

Using this comparative table of Voya's features alongside the four previously evaluated travel applications, we gain a clearer understanding of Voya's capabilities and how it differentiates itself in the travel-planning market. The selected features focus on elements most relevant to group travel coordination, real-time collaboration, budgeting, and efficient itinerary planning.

Comparative Feature	Voya	Wanderlog	Tripadvisor	Wego	Triplt
Group Itinerary Planning	✓	✓	✗	✗	✗
Shared Budget Tracking	✓	Limited	✗	✗	✗
Activity Voting / Polling	✓	✗	✗	✗	✗
Live Location Sharing	✓	✗	✗	✗	✗
Real-Time Notifications	✓	✓	✗	✗	Limited
Map-Based Trip Planning	✓	✓	Limited	✗	✓
Collaborative Editing	✓	✓	✗	✗	Limited

Reservation Management	✓	✓	✗	✓ (booking only)	✓
Travel Discovery Tools	Limited	✓	✓	Limited	✗
Travel Booking / Price Comparison	Limited	✗	Limited	✓	✗
Exportable Trip Itineraries	✓	✓	✗	✗	✓
Ideal for Group Travel	✓	✓	✗	✗	✗

6.3 Strengths, Weaknesses, and Differentiators of Voya

Strengths

1. Comprehensive Group Collaboration

Voya stands out by offering the most complete set of group-focused features among all the platforms compared. It includes shared budgeting and expense tracking, activity voting and decision-making tools, live location sharing for group members, and real-time notifications for reminders and updates. Notably, none of the competitor apps provide all four of these capabilities together, which shows Voya's unique value in the travel-planning market.

2. A Unified Trip Management Experience:

Voya eliminates the need to switch between multiple apps while planning a trip, reducing reliance on group chats, spreadsheets, maps, voting tools, and budget calculators. This consolidation greatly increases user convenience and minimizes the friction typically involved in coordinating travel, especially when multiple parties are involved.

3. Real-Time Features:

One standout feature that sets Voya apart from competitors like Triplt, Wanderlog, Wego, and Tripadvisor is its support for live collaborative editing, dynamic activity feeds, and real-time communication during travel. These capabilities enhance group safety, synchronicity, and overall coordination throughout the trip.

4. Group-Centric Design:

While most travel apps are designed primarily for solo travelers or travel agents, Voya is specifically optimized for friend groups, student trips, family vacations, and other multi-person travel experiences.

Weaknesses

1. Limited Discovery and Review Data

Compared to Tripadvisor's extensive database, Voya currently offers more limited activity-discovery tools and does not host user reviews for attractions or hotels. However, these features could be added in future updates to further enhance the app.

2. No Built-in Flight or Hotel Booking

Wego excels in price comparison and booking, whereas Voya focuses solely on planning and coordination.

3. Higher Dependency on External APIs

Voya is heavily reliant on the Google Maps API, notification services, and database services, which can be costly and introduce reliability and integration challenges.

Differentiators

1. Only Platform With Full Group Coordination Pipeline

Compared to its competitors, Voya is the only application that brings together itinerary creation, budgeting, voting, live location sharing, map integration, and real-time notifications all in one platform.

2. Group Decision-Making Tools

No competitor offers an in-app structured voting system for choosing activities, destinations, or daily plans. This feature is unique to Voya.

3. Real-Time Travel Dynamics

Features such as live location sharing and instant notifications set Voya apart from platforms like Triplt, Tripadvisor, and Wego.

4. Collaboration-Centric Planning

While Wanderlog supports collaboration, Voya is unique in the sense that it expands it by offering:

- Detailed budgeting
- Group polls
- Real-time trip status
- Member-specific reminders

5. Tailored for Social Travel

Most competitors focus on professionals or solo travelers, while Voya is designed specifically for student groups, friend groups, families, and group trips in general. It fills a major gap in the market, which is one we believe positions it strongly for success.

7. Conclusion and Evaluation

7.1 Summary of Progress Since Deliverable #1

Since Deliverable #1, progress has moved smoothly. Tasks were delegated across team members in the Team Project Proposal. All tasks for Deliverable #1 were completed on time, so feedback was taken into account. This included modifications to functional and non-functional requirements. In addition, since Deliverable #1, all remaining tasks designated for Deliverable #2 have been completed in a timely manner by all members and are displayed in the above sections of this Deliverable. This includes scheduling, cost, effort, and pricing estimation, duration and staffing, a test plan and test cases, comparison to other designs, and a presentation of the project. All of this has led to the successful completion of the project by fulfilling all required tasks.

7.2 Deviations from Original Plan and Reasons

The team received positive feedback on the proposal, so there were a few modifications to our original Team Project Proposal. We worked afterwards to expand our ideas, refine them, and address potential bottlenecks. After Deliverable #1, additional feedback emphasized requirements to be atomic, so functional and non-functional requirements were revised to adhere to the

defined standards. Otherwise, no further deviations or modifications were made to the original plan since its delivery.

7.3 Lessons Learned and Challenges Faced

From completing this deliverable, our team was able to learn a lot about how to go through the many steps involved in a project, including planning, estimating, and figuring out how to build a real project. One lesson we learned was how important it is to work on certain parts by breaking them up into multiple sections, because that allows us to split up our effort equally. By doing this, our team was able to better understand how the different parts of our app worked together, such as the backend, frontend, etc. Our feedback after the first deliverable said to focus on being more specific and consistent, so in this deliverable, we made sure to focus on improving our functional and non-functional requirements so that they were clearly testable. One challenge our team encountered was selecting the correct estimation model. Originally, Function Point Analysis seemed correct, but later, we decided that the Application Composition Model better matched our system's structure. We realized this after we did some calculations. In the testing section, our team had encountered an issue with building meaningful unit tests because lots of the classes depended on each other and needed helper classes before the Budget Summary unit could be correctly tested. The biggest thing is that our team was able to work around all these issues with clear communication and teamwork.

7.4 Future Work and Recommendations

In the future, there are many things our team could do to enhance Voya's features. With the rise of Artificial Intelligence, Voya could implement a chatbot that could answer questions, give trip suggestions, and more! Voya could also work to build in ways for customers to book flights, hotels, and more. This way, Voya could be a one-stop shop for all of a customer's travel needs. We can also expand the budgeting tools by introducing features that would automatically help the customers save money, link to their digital wallets, and help categorize the different expenses they're making. Another future idea is to implement an

“offline mode”. This is because travelers often lose connection while they are abroad, so adding an offline way to still access itineraries, locations, and expenses would all be helpful. Lastly, a future idea could also be a group chat or messaging system within the app. This way, all communication related to trips could be in one place, making it easier for the users to find what they need.

8. References

- [1] Easy Solution, "How To Use TripAdvisor To Plan A Trip Tutorial," *YouTube*, Apr. 6, 2024. [Video]. Available: <http://www.youtube.com/watch?v=Pdu-T6jjeqY>. [Accessed: Nov. 17, 2025].
- [2] Triplt, "A single (free) itinerary created for you in seconds." [Online]. Available: <https://www.tripit.com/web/free>. [Accessed: Nov. 17, 2025].
- [3] P. Xu, "Is There a Free Trip Planner?" *Wanderlog blog*, Oct. 14, 2024. [Online]. Available: <https://wanderlog.com/blog/2024/10/14/is-there-a-free-trip-planner/>. [Accessed: Nov. 17, 2025].
- [4] Wego, "Wego Mobile App for IOS & Android - The Best Travel App In the App Store & Play Store." [Online]. Available: <https://company.wego.com/app/>. [Accessed: Nov. 17, 2025].

9. Appendices

9.1 Presentation Slides Link

 **Course Team Project Deliverable #2 Presentation**

9.2 GitHub Repository Link

<https://github.com/AmrrOkasheh/-3354-Voya.git>